Chapter 6. CPU scheduling. Exercises

Original problems. State assumptions, units, and intermediate reasoning. Answers are in the separate solutions PDF.

OS06-E01. Nonpreemptive scheduling

Three jobs arrive at t=0 with bursts P1=8, P2=4, and P3=2. Ignore switching costs.

1. Compute completion, waiting, and turnaround for FCFS in order P1, P2, P3 and for nonpreemptive SJF.
2. Compare mean waiting and turnaround.
3. What information assumption supports the SJF conclusion?

Check your approach. Align every metric with its process identity and time unit.

OS06-E02. Round Robin

P1, P2, P3 arrive at zero with bursts 5,2,3; the initial ready queue is P1, P2, P3. No I/O or switching costs. A task whose quantum expires before completion joins the tail. Use q=2.

1. Give the timeline.
2. Compute each completion, waiting, and response time.
3. What changes when q=1?

Check your approach. Nonoverlapping intervals must account for all ten execution units, plus any explicitly included overhead.

OS06-E03. Priorities and starvation

High-priority jobs keep arriving while a lower-priority batch job L remains ready. Priorities range from 0 to 10, with larger values preferred. L starts at 0 and gains one priority level per 2 ms of waiting. Equal priorities use FIFO; new arrivals cannot jump ahead, and each selected job holds the CPU for at most B ms. Switching is free.

1. Construct an infinite execution in which L starves. Can one observed long wait establish starvation?
2. Explain how aging helps under the stated queue rules.
3. At the time L reaches priority 10, q ready jobs stand ahead of it and the running job has at most B ms left. Derive a bound from time zero to L's first execution. What additional load bound gives a common bound for all inputs?

Check your approach. Mean performance and a fairness guarantee are different claims.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

OS06-E04 HRTICK CFS event reasoning

One-CPU Linux 3.2 CFS has latency 6 ms, minimum granularity 0.75 ms, wakeup granularity 1 ms, GENTLE_FAIR_SLEEPERS=true, CONFIG_SCHED_HRTICK=y, runtime HRTICK=true, DOUBLE_TICK=false, an active CPU with a high-resolution timer, zero timer-callback delay, and free switches. P1/P2 arrive at 0, P3 sleeps until 9, bursts 8/4/2. All nice 0, weight 1024, initial vruntime 0; P1 enters before P2 on equal keys. Assume no unlisted event causes a switch; ordinary ticks can occur without causing additional switches. At t=9 the P3 enqueue and hrtick_update are processed before the old timer callback; no CPU work occurs between events at the same instant.

1. Find period, ideal runtime, and vruntime increment per real ms with P1/P2 runnable.
2. Give current, other ready, min_vruntime, and switch cause at 0,3,6,9,11,12,14.
3. Derive every Gantt segment from events; do not invent a switch at 8.
4. Find P3's sleep placement and identify which mechanism first requests rescheduling under the specified event ordering.

OS06-E05 Strict periodic-tick comparison

Use E04's state, disable HRTICK, and run ordinary ticks at integer milliseconds.

1. Compare delta_exec at 3 and 4 with check_preempt_tick's first condition.
2. Derive Gantt and completions.
3. Why does ideal_runtime=3 not guarantee switches exactly at 3,6,9?
4. Restore HRTICK and wake P3 at 8.5. Which old interval first changes?

OS06-E06 CPU fairness and synchronization

A/B share a CFS runqueue, weights 1024/335, period 6 ms. B initializes a buffer once, then writes ordinary non-atomic ready=1. A checks ready by ordinary load when scheduled. The buffer remains alive through A’ s read; after publishing, B does not concurrently overwrite it without synchronization.

1. Compute slices and CPU shares.
2. Does higher weight replace exclusion? Give a two-writer counterexample.
3. Which publication contract allows A to safely observe buffer after ready=1?
4. Which guarantees does a mutex around counter++ provide? Is its publication contract automatic from its name?
5. Distinguish barriers, atomic operations, mutexes, and semaphores in exclusion and visibility.

OS06-E07 RM bounds, event-by-event EDF, and short deadlines

One immediately preemptible CPU; two tasks release synchronously from 0. No blocking, suspension, I/O or overhead. C is both actual and worst-case demand. Set A: τ_1=(C=2, T=D=5), τ_2=(C=4, T=D=8). Retain older same-task jobs. EDF ties retain current, otherwise task/job order. Completion at a deadline succeeds; late jobs are retained.

1. Compute U and the two-task RM sufficient bound. Can this alone decide RM success/failure? Iterate $w^{(0)}=C_2$, $w^{(k+1)}=C_2+\lceil w^{(k)}/T_1\rceil C_1$.
2. Trace every RM segment in [0,10), with job completions/deadlines; check the fixed point.
3. Under EDF list choices, remaining demand and absolute deadlines at releases/completions in [0,16), especially5,8,10,15. Check all jobs due by 16. Which theorem certifies the entire sequence beyond this finite prefix?
4. Set B has τ_1=(1,2,2), τ_2=(2,4,4). Trace one common period under RM and prove exceeding the sufficient bound does not imply failure.
5. Set C has two tasks each C=2, T=10, D=3. Use interval demand at length 3 to prove every policy fails. Why does U certify ideal EDF for A but not C? Does A's original certificate survive added lock blocking without more analysis?

OS06-E08. An arrival need not preempt

One CPU, no I/O, zero switching cost. Jobs (arrival, demand) in ms are P1=(0,7), P2=(2,3), P3=(4,1). SRTF preempts only for a strictly shorter remaining demand; ties keep current, otherwise use job ID.

1. Trace SRTF and nonpreemptive SJF.
2. In P1/P2/P3 order, compute completion, turnaround, waiting and first-CPU response.
3. Is there a switch at 4? Compare mean waiting and explain why arrival alone does not imply preemption.

OS06-E09. Interrupted MLFQ budgets

One CPU. New jobs enter Q0: RR q=2; Q1:RR q=4; Q2:FCFS. A higher level preempts; an equal-level arrival does not. Demote only after exhausting the budget without finishing. After higher-level preemption, return to the original queue head with unused budget preserved. No I/O, boosts or costs. Jobs (arrival, demand) are A=(0,7), B=(1,2), C=(5,1), in ms.

1. List execution level, remaining demand and budget after each interval, especially at 5,6,9.
2. Compute completion, waiting and first-CPU response.
3. What changes if A receives a fresh Q1 budget of 4 at 6? Why is that a different policy?

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 6. Linux 3.2 scheduler source; Linux 3.2 scheduler features.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study